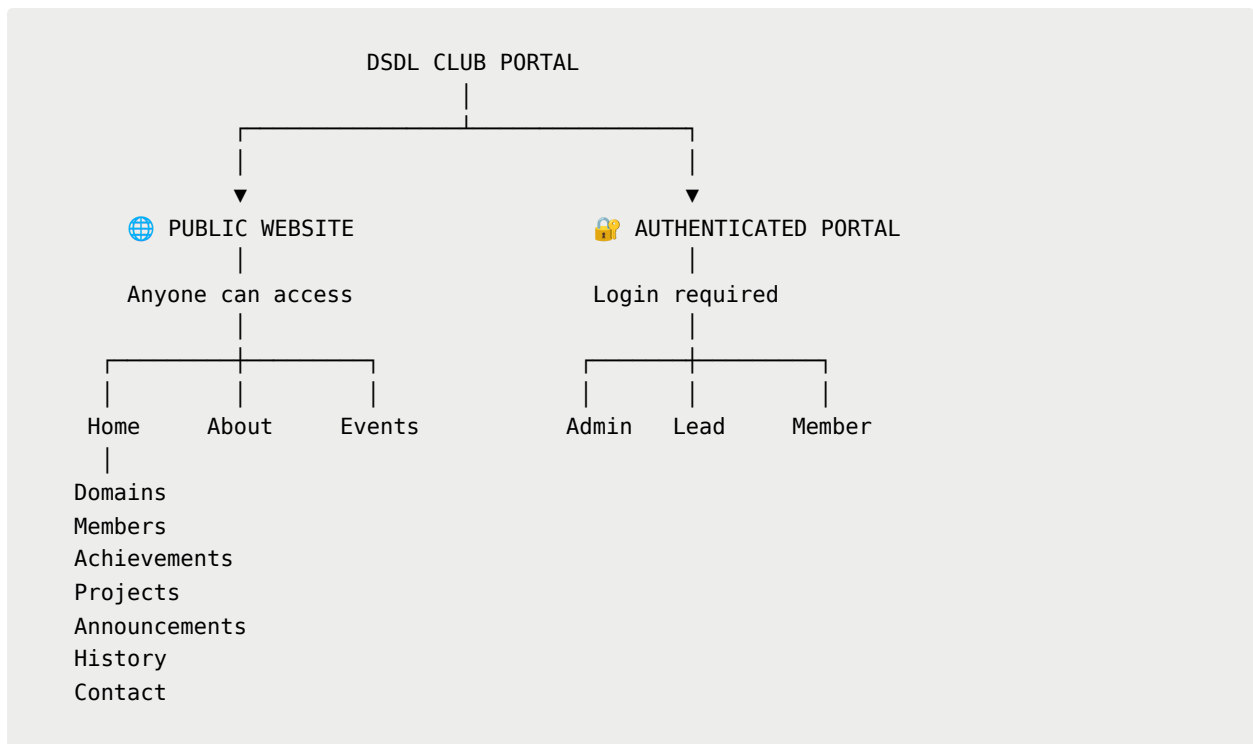


dSDL portal

i have to make a website poratal for my tech club - dSDL

1. Overall frontend architecture

Think of the frontend as **two experiences inside one React application**:



2. The PUBLIC WEBSITE

This should be the first thing anyone sees.

DSDL Logo

Home

About

Domains

Events

Achievements

Projects
Members

And on the right:(small)

 Login/ signup

3. Home Page

Section order

```
HOME
|
|— Navbar
|
|— Hero
|
|— Club Stats
|
|— What is DSDL?
|
|— What We Do
|
|— Domains
|
|— Upcoming Events
|
|— Announcements( running announcements , also in points under that )
|
|— Achievements
|
|— Featured Projects
|
|— Meet the Community
|
|— Join / CTA
|
|— Footer
```

4. Hero section

hero uses:

- white background
- subtle technical grid
- DSDL + KIET branding
- large bold headline
- blue emphasis
- blue primary CTA
- white secondary CTA

We should preserve that language.

the portal hero could be conceptually:

DSDL TECHNICAL CLUB

Build. Learn. Innovate.

A machine learning , web development and GenAI community
where students learn, build and collaborate
on real-world technology.

[Explore the Club]

[View Events]

And visually:

DSDL / KIET branding

BUILD. LEARN. INNOVATE.

Machine Learning • GenAI • Data Science

[Explore the Club]

Visual treatment

Use:

```
background: white
technical grid: extremely subtle
blue typography accents
large black/navy headings
blue CTA
tiny orange accent only where useful
```

The orange from the KIET logo can become an **accent**, not a major brand color.

5. Club Stats section

Right below the hero:

```
CLUB AT A GLANCE

80+
Members

10+
Projects

15+
Events

7+
Technical Domains
```

But these should eventually come from your backend.

Conceptually:

```
stats = {
  members: 80,
  projects: 12,
  events: 15,
  achievements: 20
}
```

Then React renders them.

That means when Admin adds a member, your public website doesn't need to be manually edited.

6. About DSDL

This should answer:

| What exactly is DSDL?

Structure:

ABOUT DSDL

Who We Are

Short paragraph.

Then:

OUR STORY

with a timeline.

Example:

```
2025
|
├─ DSDL established
|
2026
|
├─ ML initiatives
├─ GenAI workshops
├─ Technical events
├─ Student projects
└─ Community expansion
```

And:

OUR MISSION

Learn
Build
Collaborate
Innovate

The blueprint expects the public website to provide basic club information and describe the club's domains and activities.

7. Domains Page

This is particularly important for DSDL.

“Areas You Can Explore”. For the portal, I'd make that a proper domain system.

Page:

/domains

UI:

EXPLORE OUR DOMAINS

Learn. Build. Experiment. Specialize.

Machine
Learning

Generative AI

Data Science

Deep Learning

Computer Vision

NLP / LLMs

You can later have domains such as:

- Machine Learning
- Generative AI
- Data Science
- Deep Learning
- Computer Vision
- NLP / LLM
- MLOps
- AI Research

But don't hardcode these until your club confirms the actual domains.

The blueprint treats domains as database-backed entities with leads and members.

8. Domain Detail Page

Click:

Machine Learning

and get:

Machine Learning

About the Domain

Domain Lead
[Profile]

Members
[Member] [Member] [Member]

Projects
[Project Cards]

Activities
[Activity Cards]

Achievements
[Achievement Cards]

This is where the public frontend begins connecting naturally to your backend data model.

9. Events Page

Route:

/events

Have tabs or filters:

All
Upcoming
Past
Hackathons
Workshops
Bootcamps
Seminars
Competitions

The blueprint specifically includes hackathons, workshops, seminars, bootcamps, competitions, meetings and other activities as event types.

Event card

IMAGE

AI WORKSHOP
24 AUG 2026

Introduction to RAG

[View Event →]

10. Event Details Page

Route:

```
/events/:eventId
```

Contains:

Event Banner

AI AGENTS WORKSHOP

Date

Time

Venue

Organizer

Registration Deadline

About Event

Schedule

Speakers / Mentors

Participants

Gallery

Projects produced

Later, logged-in members could see:

```
[ Register for Event ]
```

The backend can then manage participants.

11. Hackathons

I would make Hackathons visually prominent.

Instead of merely:

```
Events → Hackathon
```

I'd have:

```
/events  
  ↓  
filter = Hackathons
```

and perhaps a homepage feature:

```
FEATURED HACKATHON  
  
HACKGENOME 2026  
  
Build something extraordinary.  
  
[ Explore ]
```

A hackathon detail page can eventually show:

```
Overview  
Problem Statements  
Timeline  
Rules  
Teams  
Submissions  
Winners  
Gallery
```

The team-formation functionality in the blueprint naturally connects to this. Members can create teams, specify required skills, and accept/reject join requests.

12. Announcements

Route:

```
/announcements
```

Homepage can show the latest 3–5.

Example:

LATEST ANNOUNCEMENTS

- GenAI workshop registrations open
20 August 2026
- New ML domain mentorship cycle
17 August 2026
- Hackathon team formation open
15 August 2026

Click:

```
/announcements/:id
```

The blueprint supports announcement creation/edit/delete/publish functionality for authorized users, with public visibility according to the configured rules.

13. Achievements Page

This should be visually impressive.

Route:

```
/achievements
```

Possible layout:

ACHIEVEMENTS

Celebrating the people building with DSDL.

2026



Hackathon Winner
XYZ Hackathon



Competition Finalist
ABC Challenge



12 Student Projects



15+ Workshops Conducted

Then filters:

All
Club
Hackathons
Projects
Competitions
Certifications
Members

The blueprint includes achievements such as hackathon wins, project contribution, event volunteering and competition participation.

14. Projects Page

Route:

/projects

This is a great opportunity to show that DSDL is **not just a club that conducts events—it builds things.**

Cards:

PROJECT IMAGE

AI Resume Analyzer

GenAI • NLP • RAG

Built by: Team DSDL

[View Project]

Project detail:

Problem

Solution

Technology

Architecture

Team

GitHub

Demo

Achievements

The blueprint includes Projects as one of the major profile/activity entities.

15. Members Page

Route:

/members

This is where your public website becomes a **community directory**.

Filters:

All
Domain
Lead
Member

Card:

[PHOTO]

Gauri Mishra
AI/ML

Machine Learning
GenAI
Python

GitHub LinkedIn

The blueprint defines public/member profiles around name, photo, email/phone where applicable, domain, role, skills, bio, social links, projects, achievements, hackathons, events, certifications and joining date.

For security/privacy, the **public frontend should only expose fields the club intentionally marks public.**

16. Member Profile Page

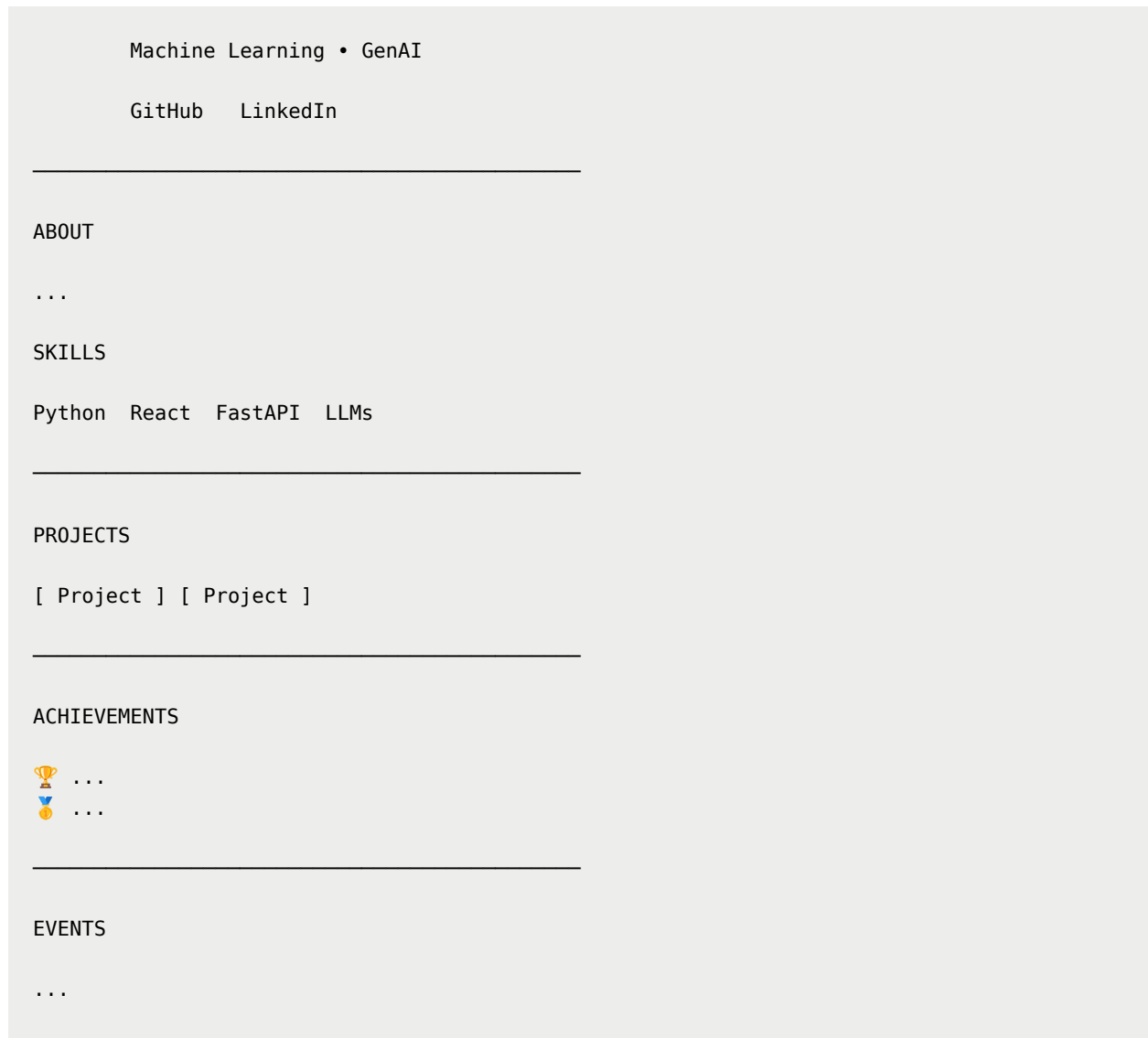
Route:

/members/:id

Visual:

[PROFILE PHOTO]

Gauri Mishra
AI / ML



17. Login Page

Route:

```
/login
```

Minimal.

Don't make it look like a generic Bootstrap form.

I'd do:

Welcome Back

DSDL Technical Club Portal

Email

[_____]

Password

[_____]

[Login]

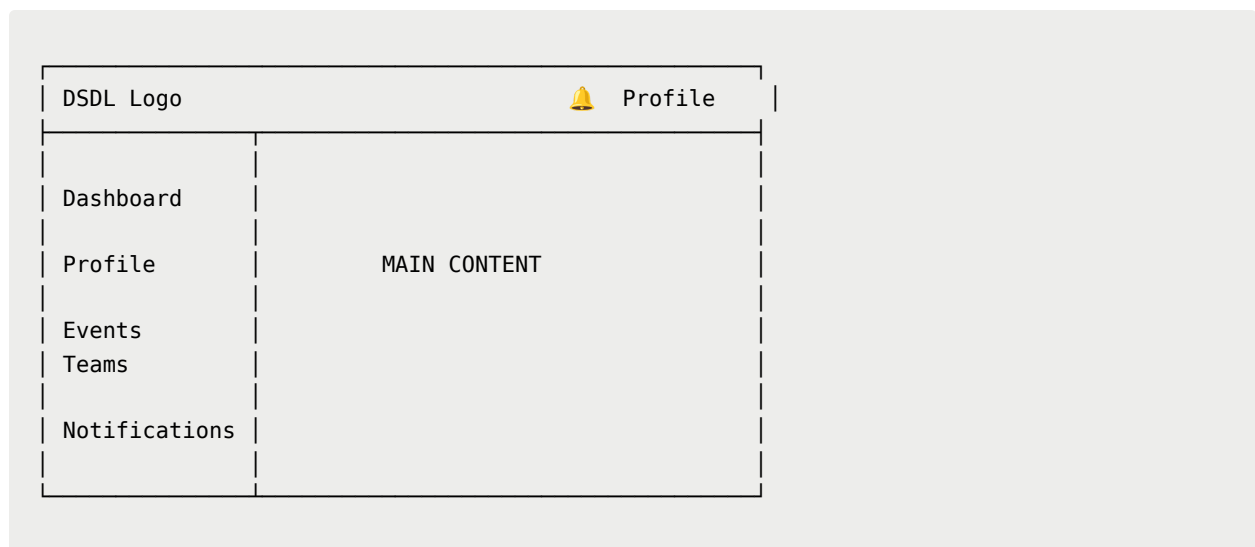
Forgot password?

Maybe subtle DSDL visual on the left or background.

18. Authenticated frontend

Now React switches into a completely different UI system.

I recommend **a dedicated dashboard layout**.



19. Member Dashboard

/member/dashboard

Sections:

Welcome, Gauri 🙌

Upcoming Events

My Teams

My Requests

My Achievements

My Projects

Notifications

Profile Completion

The blueprint gives Members access to profile management, announcements, events, registration, teams, requests, notifications and activity.

20. Lead Dashboard

/lead/dashboard

Only domain-level information.

Example:

AI/ML DOMAIN

Members	24
Projects	8
Events	6
Announcements	3

Domain Members

Projects

Activities

Announcements

Events

A Lead should not see Admin-level functionality.

21. Admin Dashboard

/admin/dashboard

This is effectively your internal control center.

Top metrics

Members 80	Leads 7	Domains 8	Events 15
Announcements 12	Teams 18	Pending Requests 6	

Then:

Members
Domains
Announcements
Events
Teams
Achievements
Projects
Activity

The blueprint explicitly gives Admins complete portal control and lists these management areas.

22. Team Formation UI

This is an **authenticated-only feature**.

Route:

```
/teams
```

Example:

FIND A TEAM

AI Hackathon

Need: 2 Members

Skills: React, Python, ML

2 / 4 members

[Request to Join]

Create team:

Create Team

Team Name

Event

Members Required

Skills Required

Description

[Create Team]

Then creator:

Join Requests

Rahul

Python

[Accept] [Reject]

Ananya

React
[Accept] [Reject]

That follows your blueprint almost exactly.

23. Notifications

Authenticated navbar:



Click:

Notifications

- You received a team join request
5 mins ago
- You were accepted into AI Warriors
2 hrs ago
- New GenAI workshop announced
Yesterday

Notification functionality is defined for all three roles in the blueprint.

24. Footer

Use the same visual language as your current DSDL website.

I'd keep:

DSDL
Data Science & Deep Learning Club
KIET Group of Institutions

About
Domains
Events

Achievements
Projects

GitHub
LinkedIn
Instagram
WhatsApp

© 2026 DSDL Technical Club

Dark navy footer is actually a good contrast to your white website.

Your current registration site already uses this white → dark navy transition effectively, so that visual idea is worth retaining.

25. COMPLETE ROUTE STRUCTURE

Here's the frontend route map I'd lock in:

```
PUBLIC
|
├─ /
|
├─ /about
|
├─ /domains
|   └─ /domains/:domainId
|
├─ /events
|   └─ /events/:eventId
|
├─ /announcements
|   └─ /announcements/:announcementId
|
├─ /achievements
|
├─ /projects
|   └─ /projects/:projectId
|
├─ /members
|   └─ /members/:memberId
|
└─ /login
```

AUTHENTICATED

```

|
├── /dashboard
|
├── /profile
|
├── /notifications
|
├── /teams
|   ├── /teams/create
|   └── /teams/:teamId
|
├── /my-events
└── /my-achievements

```

ADMIN

```

|
├── /admin
├── /admin/members
├── /admin/domains
├── /admin/events
├── /admin/announcements
├── /admin/projects
├── /admin/achievements
├── /admin/teams
└── /admin/activity

```

LEAD

```

|
├── /lead
├── /lead/members
├── /lead/events
├── /lead/announcements
├── /lead/projects
└── /lead/activity

```

26. Recommended React folder structure

Now we're getting to the part you'll actually implement.

I would structure your frontend like this:

```

frontend/
|   |   |
|   |   └── dashboard/

```

```
├─ StatCard.jsx
├─ Sidebar.jsx
├─ NotificationBell.jsx
├─ DashboardHeader.jsx
├─ pages/
│   ├── public/
│   │   ├── Home.jsx
│   │   ├── About.jsx
│   │   ├── Domains.jsx
│   │   ├── DomainDetails.jsx
│   │   ├── Events.jsx
│   │   ├── EventDetails.jsx
│   │   ├── Announcements.jsx
│   │   ├── AnnouncementDetails.jsx
│   │   ├── Achievements.jsx
│   │   ├── Projects.jsx
│   │   ├── ProjectDetails.jsx
│   │   ├── Members.jsx
│   │   └─ MemberProfile.jsx
│   ├── auth/
│   │   └─ Login.jsx
│   ├── member/
│   │   ├── Dashboard.jsx
│   │   ├── Profile.jsx
│   │   ├── MyEvents.jsx
│   │   ├── Teams.jsx
│   │   └─ Notifications.jsx
│   ├── lead/
│   │   ├── Dashboard.jsx
│   │   ├── Members.jsx
│   │   ├── Events.jsx
│   │   ├── Announcements.jsx
│   │   └─ Projects.jsx
│   └─ admin/
│       ├── Dashboard.jsx
│       ├── Members.jsx
│       ├── Domains.jsx
│       ├── Events.jsx
│       ├── Announcements.jsx
│       ├── Projects.jsx
│       ├── Achievements.jsx
│       └─ Teams.jsx
├─ context/
└─ AuthContext.jsx
```

```
|
| |
| | └─ hooks/
| |   └─ useAuth.js
| |   └─ useFetch.js
| |   └─ useDebounce.js
| |
| | └─ services/
| |   └─ api.js
| |   └─ authService.js
| |   └─ eventService.js
| |   └─ memberService.js
| |   └─ domainService.js
| |   └─ teamService.js
| |
| | └─ routes/
| |   └─ AppRoutes.jsx
| |   └─ ProtectedRoute.jsx
| |   └─ RoleRoute.jsx
| |
| | └─ utils/
| |   └─ constants.js
| |   └─ formatDate.js
| |   └─ permissions.js
| |
| | └─ styles/
| |   └─ globals.css
| |   └─ variables.css
| |
| | └─ App.jsx
| | └─ main.jsx
|
└─ package.json
```

27. Design system 🎨

This is important because your screenshots already establish a recognizable identity.

Primary

DSDL Blue

Use it for:

- primary buttons

- links
- active states
- important headings
- icons
- highlights

Secondary

Deep Navy

Use for:

- footer
- dashboard sidebar
- dark sections
- strong text

Background

White
#FFFFFF

Main site should remain predominantly white.

Surface

Very Light Blue/Grey

For cards and subtle sections.

Accent

Warm Orange

Borrowed very lightly from KIET branding.

Not dominant.

28. Visual language

From the screenshots, I'd standardize these:

Border radius

12–20px

Shadows

Very subtle.

No giant floating shadows.

Typography

Large bold headings.

Something in the style of:

Inter
Manrope
Plus Jakarta Sans

I'd personally choose **Plus Jakarta Sans** or **Inter** for this project.

Icons

Use a consistent icon library such as:

Lucide React

rather than mixing random icons.

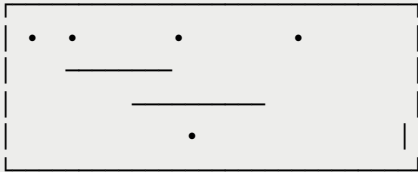
29. Technical / ML aesthetic

This is where I would differentiate your website from the registration site.

The registration website already uses a subtle grid.

Keep that.

But add **very restrained AI/technical motifs**:



Things like:

- subtle node networks
- tiny dots
- grid backgrounds
- abstract data patterns
- circuit-like lines
- animated gradient blobs

But **very subtle**.

Do NOT turn it into:

| 🤖✨ AI futuristic cyberpunk website

That's not your brand.

Your current aesthetic says:

| **professional technical community**

and I would preserve that.

30. Public vs Dashboard design

One thing I want you to understand early:

Public website

- White
- Open
- Spacious
- Marketing-oriented
- Storytelling
- Animations
- Large typography
- Photography/projects

Dashboard

- More compact
- Sidebar
- Tables
- Filters
- Forms
- Charts
- Modals
- Notifications
- Data-heavy

Don't force the same layout onto both.

They can share:

- colors
- typography
- buttons
- cards
- icons
- logo

but the **UX** should be different.

31. Frontend data architecture

Eventually your components should NOT directly call fetch everywhere.

Bad:

```
useEffect(() => {  
  fetch("http://localhost:5000/api/events")  
}, [])
```

all over your components.

Instead:

```
Component  
  ↓  
Custom Hook  
  ↓  
Service  
  ↓  
API  
  ↓  
Backend
```

For example:

```
EventPage  
  ↓  
useEvents()  
  ↓  
eventService.getEvents()  
  ↓  
GET /api/events
```

That's the kind of architecture that will keep this project maintainable when the club actually starts using it.